

TASK 05 — Production API Review, Reliability Testing & System Design

Step 1 — Review API Gateway

API Gateway

An API Gateway is the entry point through which client applications access your backend microservices.

Example, imagine an e-commerce application with three services:

- Product Service — manages product information.
- Order Service — creates and manages orders.
- Payment Service — processes payments.

Instead of the client calling each service directly, the client sends requests through the API Gateway.

Client / Browser / Mobile App

API Gateway

Routing • Filtering • Correlation ID • Error handling

Product

8081

Order

8082

Payment

8083

What should you verify

A. Routing

Routing means sending a request to the correct backend service.

Example routing rules:

Client request	Destination
----------------	-------------

/api/products/** Product Service

/api/orders/** Order Service

/api/payments/** Payment Service

If a user requests /api/products/101, the gateway should forward the request to the Product Service.

How to test: Send requests to each route and verify that the correct service responds.

B. Filtering

Filters execute additional logic before or after a request reaches a service.

Examples include:

- Authentication and authorization checks.
- Adding correlation IDs.
- Checking request headers.
- Recording request details.
- Applying rate limits.

How to test: Send a request with the required headers and another without them. Verify that the gateway applies the configured rules.

C. Correlation ID

A correlation ID is a unique identifier used to connect log entries belonging to the same request.

Example:

Request received

Correlation-ID: req-12345

Order Service processing

Correlation-ID: req-12345

Payment Service processing

Correlation-ID: req-12345

This makes it easier to investigate a request across multiple services.

D. Error handling

The gateway should handle failures consistently.

Examples:

- Unknown route → 404 Not Found.
- Invalid request → 400 Bad Request.
- Service unavailable → an appropriate 5xx response.
- Excessive requests → 429 Too Many Requests.

Confirm routing, filters, correlation ID propagation, and error handling work correctly.

Step 2 — Review Rate Limiting

rate limiting

Rate limiting controls how many requests a user or client can send within a specified time period.

For example, allow a maximum of 5 requests in 10 seconds.

- Normal requests → allowed.
- Requests exceeding the limit → rejected with HTTP 429 Too Many Requests.

This protects your services from excessive traffic and helps prevent abuse.

Example

Assume the limit is 5 requests in 10 seconds.

Request	Result
Request 1	Allowed
Request 2	Allowed
Request 3	Allowed
Request 4	Allowed
Request 5	Allowed
Request 6	Rejected with 429

The precise result depends on the rate-limiting algorithm and how the time window is configured.

Spring Cloud Gateway configuration

For a Spring Cloud Gateway application using Redis rate limiting, a route can be configured like this:

spring:

cloud:

gateway:

routes:

- id: product-service

uri: http://localhost:8081

predicates:

- Path=/api/products/**

filters:

- name: RequestRateLimiter

args:

key-resolver: "#{@ipKeyResolver}"

redis-rate-limiter.replenishRate: 5

redis-rate-limiter.burstCapacity: 5

Create the IP-based key resolver:

```
import org.springframework.cloud.gateway.filter.ratelimit
```

```
.KeyResolver;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import reactor.core.publisher.Mono;
```

```
@Configuration
```

```
public class RateLimitConfig {
```

@Bean

```
public KeyResolver ipKeyResolver() {  
    return exchange -> Mono.just(  
        exchange.getRequest()  
            .getRemoteAddress() == null  
            ? "unknown"  
            : exchange.getRequest()  
                .getRemoteAddress()  
                .getAddress()  
                .getHostAddress()  
    );  
}
```

Step 3 — Review Idempotency

idempotency

Idempotency means that repeating the same operation with the same idempotency key does not create duplicate side effects.

Imagine a customer places an order. The customer clicks the Place Order button, but the internet connection is slow. The customer clicks it again.

Without idempotency, the application might create two orders.

With idempotency, the application recognizes the repeated request and returns the original result instead of creating another order.

Example API request

POST /api/orders

Content-Type: application/json

Idempotency-Key: order-abc-123

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

The client retries the same operation using the same key.

The expected result is:

- First request: creates order 501.
- Duplicate request: returns the result for order 501.
- Database: contains only one order created by this operation.

A new, independent order should use a new idempotency key.

Spring Boot implementation example

For a simple demonstration, use an in-memory map:

```
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
import java.util.UUID;  
import java.util.concurrent.ConcurrentHashMap;  
  
@RestController  
@RequestMapping("/api/orders")  
public class OrderController {  
    private final ConcurrentHashMap<String, String> results =  
        new ConcurrentHashMap<>();  
  
    @PostMapping  
    public ResponseEntity<String> createOrder(  
        @RequestHeader("Idempotency-Key") String key,  
        @RequestBody String request) {  
        String result = results.computeIfAbsent(  

```

```

        key,

        k -> "Order created: " + UUID.randomUUID()

    );

    return ResponseEntity.ok(result);
}
}

```

Step 4 — Review Distributed Tracing

distributed tracing

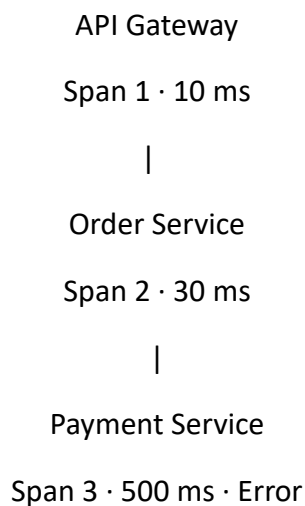
Distributed tracing helps you follow a single request as it moves through multiple microservices.

Suppose a customer places an order. The request travels through the following services:

1. API Gateway
2. Order Service
3. Payment Service
4. Inventory Service

If payment processing fails, distributed tracing helps you identify which service failed and how long each operation took.

Request flow



Illustrative timings, not measured results

A trace represents the entire request journey. A span represents an individual operation within that journey.

tracing concepts:

- Trace ID: Identifies the complete request journey.
- Span ID: Identifies one operation.
- Parent span ID: Connects a child operation to its parent.
- Duration: Shows how long an operation took.
- Status: Indicates success or failure.

Spring Boot tracing setup

For a modern Spring Boot 3 application, Micrometer Tracing with OpenTelemetry is one option.

Example Maven dependencies for a compatible Spring Boot project:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

Configure tracing in application.yml:

management:

tracing:

sampling:

probability: 1.0

otlp:

tracing:

endpoint: http://localhost:4318/v1/traces

Here, sampling probability 1.0 means recording all eligible traces. Use an appropriate sampling rate for production traffic. The OTLP endpoint must point to a running, compatible tracing collector or backend.

You can visualize traces using tools such as:

- Jaeger
- Grafana Tempo
- Zipkin

Testing procedure

1. Start the tracing backend or collector.
2. Start your Spring Boot services with tracing configured.
3. Send a request through the gateway.
4. Open the tracing UI.
5. Search for the request's trace.
6. Verify that the trace contains the relevant service spans.
7. Simulate a service failure and check whether the failed span is visible.

Step 5 — Review Structured Logging

structured logging

Structured logging means recording application events in a consistent, machine-readable format, commonly JSON.

Traditional log:

Payment failed for order 501

Structured log:

```
{  
  "timestamp": "2026-10-09T10:00:00Z",  
  "level": "ERROR",  
  "service": "payment-service",  
  "message": "Payment processing failed",  
  "orderId": "501",  
  "traceId": "abc123",  
  "errorCode": "PAYMENT_TIMEOUT"  
}
```

The JSON example is illustrative. A real application should populate the timestamp and trace ID from the actual event and tracing context.

Structured logs are easier to search, filter, analyze, and send to centralized logging platforms.

What information should logs contain

Field	Purpose
Timestamp	When the event occurred
Log level	INFO, WARN, ERROR, etc.
Service name	Which microservice generated the log
Trace ID	Which request the event belongs to
Message	What happened
Error details	What went wrong

Request or order ID Which operation is involved

Avoid logging passwords, access tokens, payment card details, or other sensitive information.

Spring Boot logging example

Use SLF4J for application logs:

```
import org.slf4j.Logger;
```

```

import org.slf4j.LoggerFactory;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/payments")

public class PaymentController {

    private static final Logger log =

        LoggerFactory.getLogger(PaymentController.class);

    @PostMapping

    public String processPayment() {

        log.info("Payment processing started");

        try {

            // Add actual payment processing here.

            log.info("Payment processing completed");

            return "Payment successful";

        } catch (Exception ex) {

            log.error("Payment processing failed", ex);

            throw ex;

        }

    }

}

```

This demonstrates the logging API, but it does not yet produce JSON automatically. Configure a JSON logging encoder or compatible logging appender if machine-readable JSON output is required.

Testing procedure

1. Start the Payment Service.
2. Submit a payment request.
3. Verify that an informational log is generated.

4. Trigger a controlled test failure.
5. Verify that an error log includes useful exception details.
6. Check that the service name and trace ID are available.
7. Verify that sensitive data is not present in the logs.
8. If using JSON logging, validate that each log entry is valid JSON

Step 6 — Create Complete Microservices Architecture

Explanation of every component

Client

The client is the application used by the customer.

Examples:

- Website
- Android or iOS application
- Frontend built using React or Angular

Example: A customer clicks Place Order on an e-commerce website.

The client sends an HTTP request to the API Gateway rather than directly calling every microservice.

API Gateway

The API Gateway is the entry point for requests to the backend.

Its responsibilities include:

- Routing requests to the correct service.
- Authentication and authorization integration.
- Rate limiting.
- Request and response filtering.
- Correlation ID propagation.
- Consistent error handling.

Example routing:

Endpoint	Destination
/api/users/**	User Service
/api/orders/**	Order Service
/api/products/**	Product Service
/api/payments/**	Payment Service, if exposed through the gateway

You can implement the gateway using Spring Cloud Gateway.

User Service

The User Service manages customer information.

Its responsibilities may include:

- Creating user profiles.
- Retrieving user details.
- Updating profile information.
- Managing user-related business rules.

Example API:

GET /api/users/101

Example response:

```
{  
  "userId": 101,  
  "name": "Ravi",  
  "email": "ravi@example.com"  
}
```

A real application must apply appropriate authorization and protect personal information.

Order Service

The Order Service manages the order lifecycle.

Its responsibilities include:

- Creating orders.
- Retrieving order details.
- Updating order status.
- Coordinating payment and inventory operations.
- Preventing duplicate order creation using idempotency.

Example API:

POST /api/orders

Content-Type: application/json

Idempotency-Key: order-abc-123

Example request:

```
{  
  "userId": 101,  
  "productId": 501,  
  "quantity": 2  
}
```

The service validates the order and coordinates the required business operations. It should not assume payment has succeeded until it receives a reliable payment result.

Product Service

The Product Service manages product catalogue information.

Its responsibilities include:

- Adding products.
- Retrieving product details.
- Updating product descriptions and prices.
- Providing product search or catalogue APIs.

Example:

GET /api/products/501

Example response:

```
{  
  "productId": 501,  
  "name": "Laptop",  
  "price": 55000  
}
```

The Product Service owns product catalogue data. Inventory quantities may be managed by a separate Inventory Service in a larger system.

Payment Service

The Payment Service processes payment operations.

Its responsibilities include:

- Initiating payments.
- Communicating with a payment provider.
- Recording payment status.
- Handling failed or pending transactions.
- Supporting safe retries and payment idempotency.

Example payment states:

PENDING → SUCCESS

or

PENDING → FAILED

A payment timeout does not necessarily mean the payment failed. The system may need to check the provider's status before retrying to avoid charging a customer twice.

Redis

Redis is an in-memory data store commonly used for caching and other fast data operations.

In this architecture, Redis can be used to:

- Cache frequently requested product information.
- Store rate-limiting counters.

- Store temporary session data.
- Support distributed idempotency when implemented with suitable atomic operations and expiration policies.

Example:

A customer requests product 501 repeatedly. Instead of querying the database for every request, the Product Service can retrieve the product from Redis when a valid cached copy exists.

Typical flow:

1. Check Redis for product 501.
2. If the product is cached, return it.
3. If not, retrieve it from the database.
4. Store the result in Redis with an appropriate expiration time.
5. Return the product to the client.

Redis is not automatically a replacement for a durable database. Critical order and payment records should have appropriate persistent storage.

Database

The database stores information that must persist.

Examples:

- User profiles.
- Product records.
- Orders and order status.
- Payment records.

In a microservices architecture, each service should own its data and control access to it. Separate databases or logically isolated schemas can help preserve that ownership.

For example:

Service	Example data
User Service	Users and profiles

Product Service Product catalogue

Order Service Orders and order items

Payment Service Payment transactions

A database may be relational, such as PostgreSQL or MySQL, depending on the service's requirements.

Apache Kafka

Apache Kafka is a distributed event-streaming platform that allows services to communicate asynchronously.

Instead of requiring one service to call every other service directly, it can publish an event that interested services consume.

Example:

After an order is successfully created, the Order Service publishes:

```
{  
  "eventType": "OrderCreated",  
  "orderId": 1001,  
  "userId": 101,  
  "status": "CREATED"  
}
```

Other services can consume this event:

- Notification Service sends an order confirmation.
- Inventory Service reserves stock.
- Analytics Service records order activity.

Kafka helps decouple these services, but consumers must be designed to handle duplicate event delivery and failures safely.

Other Services

These are additional services connected to the system through synchronous APIs or asynchronous events.

Examples include:

- Inventory Service — stock availability and reservation.
- Notification Service — email and SMS notifications.
- Shipping Service — shipment creation and tracking.
- Analytics Service — business reporting.

You do not need to build all these services initially. Start with the core services and add others when the business requirements justify them.

Step-by-step working flow

Let's follow what happens when a customer places an order.

Customer sends a request

The client sends POST /api/orders to the API Gateway with order details and an idempotency key.

Gateway processes the request

The gateway applies the configured security, routing, rate-limiting, and correlation-ID rules, then forwards the request to the Order Service.

Order Service validates the order

It checks the request, retrieves product information, verifies applicable business rules, and safely handles retries.

Payment is processed

The Order Service coordinates with the Payment Service. The payment result determines the next order state.

Order data is saved

The appropriate service stores its own records in its database. Redis may cache suitable data, but the database remains the source of truth for durable records.

Events are published

The system publishes events such as OrderCreated or PaymentSucceeded to Kafka when the corresponding state changes have been committed reliably.

Other services react

Inventory, notifications, and shipping services process relevant events according to their responsibilities.

The exact order of payment, inventory reservation, and event publication depends on your business workflow. For reliable production processing, a saga or transactional outbox pattern may be appropriate.

How to create this architecture step by step

Since this architecture uses Java and Spring Boot microservices, follow this implementation order.

Create the Spring Boot projects

Create these initial projects using Spring Initializr.

Project	Responsibility	Example port
api-gateway	Request routing	8080
user-service	User management	8081
order-service	Order management	8082
product-service	Product management	8083
payment-service	Payment processing	8084

The ports are illustrative; configure them consistently across your projects.

Useful dependencies:

- API Gateway: Spring Cloud Gateway, Actuator.
- User Service: Spring Web, Spring Data JPA, database driver.
- Order Service: Spring Web, Spring Data JPA, database driver.
- Product Service: Spring Web, Spring Data JPA, database driver.
- Payment Service: Spring Web, database driver.
- Kafka consumers/producers: Spring for Apache Kafka.
- Redis integration: Spring Data Redis.

Choose Spring Boot and Spring Cloud versions from their compatible release documentation.

Configure the API Gateway

For example, create a route to the Product Service:

server:

port: 8080

spring:

cloud:

gateway:

routes:

- id: product-service

uri: http://localhost:8083

predicates:

- Path=/api/products/**

Now a client request to:

GET http://localhost:8080/api/products/501

is forwarded to the Product Service at port 8083.

Add routes for the other services after verifying that this route works.

Implement the Product and User Services

Create REST controllers for retrieving products and users.

Example Product Service controller:

```
@RestController
```

```
@RequestMapping("/api/products")
```

```
public class ProductController {
```

```
    @GetMapping("/{id}")
```

```
    public Map<String, Object> getProduct(
```

```
        @PathVariable Long id) {
```

```
        return Map.of(
```

```
        "productId", id,  
        "name", "Laptop",  
        "price", 55000  
    );  
}  
}
```

Required imports:

```
import java.util.Map;  
  
import org.springframework.web.bind.annotation.*;
```

This is a basic demonstration. Replace the hardcoded product with a database-backed repository when implementing the complete service.

Implement the Order Service

Create an endpoint such as POST /api/orders.

The service should:

1. Validate the request.
2. Check the idempotency key.
3. Verify product and stock availability.
4. Coordinate payment.
5. Persist the order and its state.
6. Publish appropriate events reliably.
7. Return the order status.

Avoid assuming that an order is paid merely because the payment request was sent.

Connect Redis

Configure Redis for product caching or rate limiting.

Example application configuration:

spring:

data:

redis:

host: localhost

port: 6379

Start Redis and configure your application to use it. For example, Docker can run a local development instance:

```
docker run --name redis -p 6379:6379 -d redis:7
```

Use suitable cache expiration and invalidation rules so that customers do not receive stale product information.

Connect the database

Configure the relevant database for each service.

Example PostgreSQL configuration:

spring:

datasource:

url: jdbc:postgresql://localhost:5432/orderdb

username: appuser

password: \${DB_PASSWORD}

jpa:

hibernate:

ddl-auto: update

Create the database and configure the credentials before starting the service. For production, prefer schema migrations such as Flyway or Liquibase over automatic schema updates.

Connect Kafka

Configure a Kafka broker and use Spring Kafka to publish and consume business events.

For example, the Order Service publishes an OrderCreated event, and the Notification Service consumes it.

Ensure that event publication is reliable. A transactional outbox is a common solution when a database update and an event publication must remain consistent.

Add the production-readiness features

Integrate the five features from your previous task:

- API Gateway routing and error handling.
- Rate limiting.
- Idempotency.
- Distributed tracing.
- Structured logging.

Test each feature separately before running the entire architecture together.

Final testing checklist

Test	Expected result
Retrieve a product through the gateway	Product response returned
Send excessive requests	HTTP 429 when the limit is exceeded
Repeat an order request with the same key	No duplicate order
Trace a request across services	Related spans visible
Trigger a controlled service failure	Useful error logs generated
Publish an order event	Intended Kafka consumers receive it
Restart a service	Persisted records remain available

Step 7 — Test Failure Scenarios

failure scenario testing

Failure scenario testing means deliberately making a component unavailable or slow to verify that the application handles the problem safely.

For example, suppose a customer places an order, but the Payment Service is unavailable. The system should not falsely report that the payment succeeded.

The goal is to ensure that the application:

- Does not crash unnecessarily.
- Returns meaningful error responses.
- Avoids duplicate orders or payments.
- Handles timeouts correctly.
- Records useful logs and traces.
- Recovers when the failed component becomes available again.

Seven failure scenarios

Test 1 — User Service unavailable

Expected behavior: return a suitable error, such as HTTP 503, if the service cannot be reached. Do not return fake user data.

Test 2 — Payment Service unavailable

Expected behavior: keep the payment status pending or unknown until verified. Do not mark the order as paid without confirmation.

Test 3 — Redis unavailable

Expected behavior: use a controlled fallback where appropriate, or return an error if Redis is essential to the operation.

Test 4 — Kafka unavailable

Expected behavior: handle event publishing failures reliably; do not silently lose important order events.

Test 5 — Excessive API requests

Expected behavior: the rate limiter rejects requests beyond the configured limit with HTTP 429.

Test 6 — Duplicate order request

Expected behavior: a retry with the same idempotency key must not create another order.

Test 7 — Slow downstream service

Expected behavior: apply timeouts and appropriate failure handling instead of waiting indefinitely.

Test User Service unavailable

Assume the User Service runs on port 8081.

Step 1: Check that the service is running

```
curl -i http://localhost:8081/api/users/101
```

If the endpoint exists and the request is valid, you should receive its normal response.

Step 2: Stop the User Service

Stop the Spring Boot process using its terminal's Ctrl+C, or stop the relevant container if it runs in Docker.

Step 3: Send the request through the API Gateway

```
curl -i http://localhost:8080/api/users/101
```

Step 4: Verify the result

The gateway should return an appropriate failure response, commonly 503 Service Unavailable or a configured 502 Bad Gateway, depending on how the failure is handled.

The important point is that it should not hang indefinitely or return a fake successful response.

Step 5: Restart the service

Start the User Service again and repeat the request.

Expected result: the API recovers and returns the normal response once the service is available.

Test Payment Service unavailable

Assume the Payment Service runs on port 8084.

1. Start the Order and Payment Services.
2. Submit a valid order.
3. Stop the Payment Service.
4. Submit another order through the API Gateway.
5. Check the response and the stored order status.
6. Verify that the order is not incorrectly marked as paid.
7. Restart the Payment Service.
8. Verify that pending operations can recover or be reconciled safely.

Example request:

```
curl -i -X POST http://localhost:8080/api/orders \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: test-order-1001" \  
-d '{"userId":101,"productId":501,"quantity":1}'
```

The response depends on your workflow. The order may remain pending, or the application may return an error. Both can be valid if the state is consistent and retries are safe.

Important: If a payment request times out, verify its actual status with the payment provider before initiating a new charge.

Test Redis unavailable

Assume Redis runs on port 6379.

Step 1: Check Redis

```
redis-cli -p 6379 PING
```

Expected output when Redis is running:

PONG

Step 2: Stop Redis

If you started Redis using Docker:

```
docker stop redis
```

Step 3: Send an API request

```
curl -i http://localhost:8080/api/products/501
```

Step 4: Verify the fallback

The result depends on how Redis is used:

- If Redis is only a product cache, the Product Service may fetch the product from the database.
- If Redis provides a required rate-limiting or idempotency function, the application may need to reject the request or use another safe mechanism.
- The application should log the failure and avoid returning incorrect data.

Step 5: Restart Redis

`docker start redis`

Repeat the request and verify that normal behavior resumes.

Test Kafka unavailable

Kafka is used for asynchronous communication between services.

Step 1: Start Kafka and your services

Confirm that the Order Service can publish events and the consumers can receive them.

Step 2: Stop Kafka

If Kafka is running in a Docker container, identify the correct container:

`docker ps`

Stop the Kafka container by its actual name:

`docker stop <kafka-container-name>`

Replace the placeholder with your container name.

Step 3: Create an order

Send a valid order request and observe what happens when the application tries to publish an event.

Step 4: Verify event reliability

Check that:

- A publication failure is logged.
- An important event is not silently discarded.
- The order and event state remain consistent.
- A retry or recovery mechanism exists.

A common production approach is the transactional outbox pattern. The service stores the order and an event record in the same database transaction, then publishes the event to Kafka asynchronously.

Step 5: Restart Kafka

Start Kafka again and verify that pending events are published and consumed.

Expected result: the system recovers without losing important business events.

Test excessive API requests

This test verifies rate limiting at the API Gateway.

Assume the gateway allows five requests within the configured window.

Step 1: Send multiple requests

In a Bash-compatible terminal, use:

```
for i in {1..10}; do
  curl -s -o /dev/null \
    -w "Request $i: HTTP %{http_code}\n" \
    http://localhost:8080/api/products/501
done
```

Step 2: Review the HTTP status codes

Example output, assuming the limit is five requests and the requests fall within the same rate-limit window:

Request 1: HTTP 200

Request 2: HTTP 200

Request 3: HTTP 200

Request 4: HTTP 200

Request 5: HTTP 200

Request 6: HTTP 429

...

Actual results depend on the configured rate-limiting algorithm, existing traffic, and the time window.

Step 3: Verify the result

- Requests within the limit should be processed.
- Excess requests should receive 429 Too Many Requests.
- Requests should be allowed again when the applicable limit permits it.

Test duplicate order requests

This test checks the idempotency implementation.

Step 1: Submit the first request

```
curl -i -X POST http://localhost:8080/api/orders \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: order-test-501" \  
-d '{"userId":101,"productId":501,"quantity":1}'
```

Record the returned order ID.

Step 2: Submit the exact same request again

Use the same URL, request body, and idempotency key.

Step 3: Check the response

The service should return the original operation's result rather than create another order.

For example:

```
{  
  "orderId": 501,  
  "status": "CREATED"  
}
```

Both requests should refer to the same order. The precise response status may be 200 OK on a replay or the original status, depending on your API contract.

Step 4: Check the database

Verify that only one order exists for this idempotency key.

Step 5: Test a different key

Send the request using a new key. It should create a new order if the request is otherwise valid.

Expected result: repeated requests with the same key do not cause duplicate order creation or duplicate charges.

Test a slow downstream service

A downstream service is a service called by another service. For example, the Order Service may call the Payment Service.

Step 1: Introduce a delay in a test environment

For a simple test, temporarily add a delay to a development endpoint:

```
@GetMapping("/test/slow")  
  
public String slowEndpoint() throws InterruptedException {  
    Thread.sleep(5000);  
    return "Completed";  
}
```

This deliberately waits five seconds. Use only a controlled development or test environment.

Step 2: Configure a client timeout

For example, configure the HTTP client used by the Order Service to stop waiting after an appropriate period, such as two seconds.

Step 3: Send a request

Call the endpoint through the gateway and measure the result.

Step 4: Verify the behavior

- The caller should not wait indefinitely.
- The timeout should be logged.
- The application should return a meaningful error or pending status.
- Retrying should not duplicate a payment or order.
- A circuit breaker may temporarily stop repeated calls to a failing service.

Remove the delay

Restore the endpoint and verify that the application works normally again.

Step 7 completion check: All seven scenarios have been tested, failures are handled predictably, and recovery has been verified.

Step 8 — Check API Responses

What is an HTTP status code?

An HTTP status code tells the client whether its request succeeded, was invalid, was unauthorized, or failed because of a server-side problem.

Explanation of all status codes

200

OK

The request succeeded.

Example: GET /api/products/501 returns the product.

201

Created

A new resource was created successfully.

Example: POST /api/orders creates an order.

400

Bad Request

The request is malformed or its input is invalid.

Example: Quantity is zero when the API requires a positive quantity.

401

Unauthorized

Valid authentication credentials are missing or invalid.

Example: A protected endpoint receives no valid access token.

403

Forbidden

The user is authenticated but is not allowed to perform the action.

Example: A customer tries to access an administrator-only endpoint.

404

Not Found

The requested resource or endpoint does not exist.

Example: GET /api/products/999999 when that product does not exist.

409

Conflict

The request conflicts with the current resource state.

Example: An order cannot be cancelled because it has already shipped.

429

Too Many Requests

The client has exceeded a configured rate limit.

Example: A client sends too many API requests within the allowed window.

500

Internal Server Error

An unexpected server-side error occurred.

Example: An unhandled exception occurs while processing a request.

503

Service Unavailable

The service cannot handle the request temporarily.

Example: The application is unavailable during maintenance or overload.

Important difference: 401 means authentication is missing or invalid; 403 means the request is understood but access is not permitted. 500 indicates an unexpected server failure, whereas 503 indicates temporary unavailability.

Test API responses using Postman or curl

Test 1 — HTTP 200 OK

Send a valid GET request:

```
curl -i http://localhost:8080/api/products/501
```


Expected: 200 OK if the product exists and the request succeeds.

Test 2 — HTTP 201 Created

Send a valid order-creation request:

```
curl -i -X POST http://localhost:8080/api/orders \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: order-test-100" \  
-d '{"userId":101,"productId":501,"quantity":1}'
```

Expected: 201 Created if your API creates a new order synchronously.

Test 3 — HTTP 400 Bad Request

Send invalid data:

```
curl -i -X POST http://localhost:8080/api/orders \  
-H "Content-Type: application/json" \  
-d '{"userId":101,"productId":501,"quantity":0}'
```

Expected: 400 Bad Request if your application validates that quantity must be positive. You must implement this validation; Spring Boot does not automatically enforce that business rule.

Test 4 — HTTP 401 Unauthorized

Send a request to an endpoint that requires authentication without valid credentials.

```
curl -i http://localhost:8080/api/admin/users
```

Expected: 401 if authentication is required and no valid credentials are supplied. Configure Spring Security and the endpoint's security rules first.

Test 5 — HTTP 403 Forbidden

Authenticate as a user who lacks the required permission, then call an administrator-only endpoint.

Expected: 403 Forbidden, provided the application is configured to distinguish insufficient permissions from authentication failure.

Test 6 — HTTP 404 Not Found

Request a nonexistent resource:

```
curl -i http://localhost:8080/api/products/999999
```

Expected: 404 Not Found if the endpoint exists but the requested product does not. Implement this behavior in the controller or service.

Test 7 — HTTP 409 Conflict

Attempt an operation that conflicts with the current business state, such as cancelling an already-shipped order.

Expected: 409 Conflict if that is the status defined by your API contract.

Test 8 — HTTP 429 Too Many Requests

Repeat a request rapidly until you exceed the gateway's configured rate limit.

Expected: 429 Too Many Requests.

Test 9 — HTTP 500 Internal Server Error

In a controlled test environment, trigger an unexpected server exception.

Expected: 500 Internal Server Error unless the exception is deliberately mapped to another status. The response should not expose stack traces or secrets.

Test 10 — HTTP 503 Service Unavailable

Stop a required downstream service and send a request that depends on it.

Expected: 503 Service Unavailable if your application maps that failure to 503. A gateway might instead return 502 for some upstream connection failures, so document and test your chosen behavior.

Implement proper error responses in Spring Boot

Create a global exception handler in your service so that expected failures return appropriate HTTP status codes.

```
import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestControllerAdvice

public class GlobalExceptionHandler {
```

```
@ExceptionHandler(IllegalArgumentException.class)

public ResponseEntity<Map<String, String>> handleBadRequest(

    IllegalArgumentException ex) {

    return ResponseEntity

        .badRequest()

        .body(Map.of(

            "error", "BAD_REQUEST",

            "message", ex.getMessage()

        ));

}

@ExceptionHandler(ResourceNotFoundException.class)

public ResponseEntity<Map<String, String>> handleNotFound(

    ResourceNotFoundException ex) {

    return ResponseEntity

        .status(HttpStatus.NOT_FOUND)

        .body(Map.of(

            "error", "NOT_FOUND",

            "message", ex.getMessage()

        ));

}

@ExceptionHandler(Exception.class)

public ResponseEntity<Map<String, String>> handleUnexpectedError(

    Exception ex) {

    return ResponseEntity

        .status(HttpStatus.INTERNAL_SERVER_ERROR)

        .body(Map.of(
```

```
        "error", "INTERNAL_SERVER_ERROR",
        "message", "An unexpected error occurred"
    ));
}
```

Create the custom exception:

```
public class ResourceNotFoundException
    extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

For example, when a product does not exist, your service can throw:

```
throw new ResourceNotFoundException(
    "Product not found"
);
```

The global handler then returns HTTP 404.

Final verification checklist

Testing progress

0/17

Failure scenarios

User Service unavailable

Payment Service unavailable

Redis unavailable

Kafka unavailable

Excessive API requests

Duplicate order request

Slow downstream service

API responses

200 OK

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

429 Too Many Requests

500 Internal Server Error

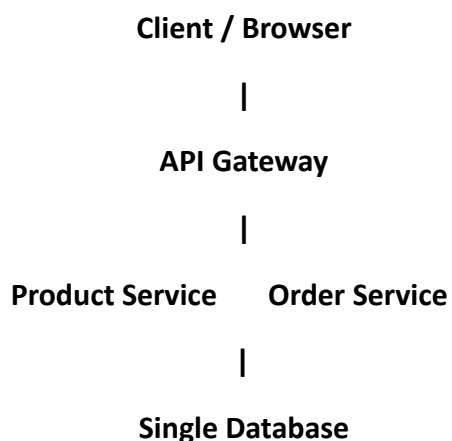
503 Service Unavailable

Step 9 — Identify Single Points of Failure (SPOF)

Single Point of Failure

A Single Point of Failure (SPOF) is a component whose failure can cause the entire application, or an important part of it, to stop working.

Example :Imagine an e-commerce application with this architecture:



If the only database fails, both services may lose access to essential data.

For example, customers may be unable to view products or place orders if the database is unavailable.

What happens if this component fails?

Identify the impact of failure.

Component	Possible impact
API Gateway	Clients may not reach backend services.
Authentication service	Users may be unable to log in.
Product Service	Product information may be unavailable.
Order Service	Customers may be unable to place orders.
Kafka	Events may stop being processed or accumulate.
Redis	Cache misses increase, and the database receives more requests.
Database	Reading or writing essential data may fail.

How does the system recover?

Decide how the application will return to normal operation.

Examples:

- API Gateway: Run multiple gateway instances behind a load balancer.
- Authentication: Use redundant instances and monitor service health.
- Kafka: Use a suitable replicated topic configuration and healthy brokers.
- Redis: Configure appropriate replication or managed high availability.
- Database: Use database replication, automated failover, and tested backups.

Recovery depends on the component. Replication and failover improve availability, while backups help recover data after certain failures.

How can we reduce the impact?

Use techniques such as:

- Redundancy: Run multiple instances of critical components.

- Load balancing: Distribute requests across healthy instances.
- Timeouts: Stop waiting indefinitely for a slow service.
- Retries: Retry temporary failures with limits and backoff.
- Circuit breakers: Temporarily stop calls to an unhealthy service.
- Monitoring and alerts: Detect failures quickly.
- Backups: Restore data after data loss or corruption.

Spring Boot services

Suppose your application has these services:

- Product Service — port 8083
- Order Service — port 8084
- API Gateway — port 8080

If the Product Service stops working, the API Gateway should detect the failure and avoid routing requests to that unhealthy instance.

If several Product Service instances are running, requests can be routed to the remaining healthy instances.

If only one Product Service instance exists, a gateway alone cannot make that service highly available.

Step 10 — Prepare a Production-Readiness Checklist

Production readiness means checking that your application is secure, reliable, observable, and prepared to handle real users and failures.

1. API Gateway

Purpose: Provides a single entry point for client requests.

It can handle routing, authentication integration, rate limiting, and other cross-cutting requirements.

Example route:

GET /api/products → Product Service

Check that routes, security rules, health checks, and failure handling are configured correctly.

2. Authentication

Purpose: Verifies who the user is.

For example, a user logs in with valid credentials and receives a session or access token.

Common technologies include Spring Security and OAuth 2.0 / OpenID Connect.

Check token validation, expiration, and secure credential storage.

3. Authorization

Purpose: Determines what an authenticated user is allowed to do.

Example:

- Customer: View products and their own orders.
- Administrator: Manage products and users.

In Spring Security, you can enforce role-based access rules.

Authentication asks, "Who are you?"

Authorization asks, "What are you allowed to do?"

4. Rate Limiting

Purpose: Restricts how many requests a client can make within a time period.

Example: Allow up to 100 requests per minute for a particular client, depending on the API's needs.

This helps reduce abuse and protects backend services from excessive traffic.

5. Idempotency

Purpose: Prevents duplicate effects when a request is repeated.

Example: A customer clicks the payment button twice. An idempotency key helps ensure the same logical payment request is not charged twice.

Use this for operations such as payment processing and order creation where duplicate actions are dangerous.

6. Timeout

Purpose: Limits how long a service waits for another service to respond.

Example: Order Service calls Payment Service and waits for at most two seconds.

If the timeout expires, the call fails instead of waiting indefinitely.

Configure connection and response timeouts based on actual service requirements.

7. Retry

Purpose: Attempts a failed operation again when the failure may be temporary.

Example: A request fails because of a brief network interruption, so the client retries it after a short delay.

Use bounded retries and exponential backoff. Retry only when safe; repeating a payment or other non-idempotent operation without protection can create duplicate effects.

8. Circuit Breaker

Purpose: Prevents repeated calls to a service that is consistently failing.

It commonly has three states:

Closed

Calls are allowed.

Open

Calls are blocked temporarily.

Half-open

Test calls check recovery.

If the test calls succeed, the circuit closes. If they fail, it opens again.

This protects your application from repeatedly waiting for a failing dependency.

Spring Boot applications commonly use Resilience4j for circuit breakers.

9. Kafka

Purpose: Supports asynchronous communication between services through events.

Example:

1. Order Service creates an order.
2. It publishes an OrderCreated event.
3. An inventory consumer processes the event.
4. A notification consumer sends a confirmation.

Check broker availability, topic replication, consumer lag, retries, dead-letter handling, and duplicate-event processing.

10. Redis

Purpose: Stores frequently accessed data in memory for fast retrieval.

Example: Product details are cached so the application does not need to query the database for every request.

Check cache expiration, memory usage, connection failures, and the fallback strategy when Redis is unavailable.

11. Database

Purpose: Stores persistent application data.

Check:

- Database connectivity and connection pool limits.
- Transaction management and data consistency.
- Indexes and query performance.
- Backup and restore procedures.
- Replication and failover, where required.
- Database credentials and access permissions.

A successful backup is not enough; regularly test whether the data can actually be restored.

12. Logging

Purpose: Records application events and errors.

Example:

```
private static final Logger log =
```

```
    LoggerFactory.getLogger(OrderService.class);
```

```
log.info("Order created successfully: {}", orderId);
```

```
log.error("Unable to process order: {}", orderId, exception);
```

Use structured logs where possible. Avoid logging passwords, access tokens, payment details, or other sensitive information.

13. Distributed Tracing

Purpose: Tracks a request as it travels across multiple services.

Example:

Client → API Gateway → Order Service → Payment Service → Database.

A trace helps you identify which service caused a slow response or error.

Common tools include OpenTelemetry, Jaeger, and Zipkin.

14. Error Handling

Purpose: Returns consistent and meaningful errors instead of exposing internal implementation details.

Example response:

```
{  
  "status": 404,  
  "error": "PRODUCT_NOT_FOUND",  
  "message": "The requested product was not found"  
}
```

Use appropriate HTTP status codes, centralized exception handling, and safe error messages.

In Spring Boot, `@RestControllerAdvice` is commonly used for global exception handling.

15. API Documentation

Purpose: Explains how developers can use your APIs.

Documentation should include:

- API endpoints and HTTP methods.
- Request and response examples.
- Authentication requirements.
- Validation rules and error responses.
- API versions and deprecation information.

Springdoc OpenAPI can generate interactive API documentation for Spring Boot applications

